



Universidad Nacional Autónoma de México
Facultad de Ciencias

Ingeniería de Software
Iteración 4

Versión del Documento: 4.0

Profesora:

Karla Socorro García Alcántara

Ayudantes:

- Maitreyi Martínez Jiménez
- Luis Gerardo Bernabe Gomez

Equipo:

Kernel Crew y La Sonora Dinamita.

Integrantes:

- Carrillo Gonzalez Luis Israel
- Flores Galeana Daniela
- González Castillo Patricio Salvador
- Guadalupe Abrego Karla Sheridam
- Raya Garcia Marco Antonio
- Rubio Resendiz Marco Antonio

Índice

Introducción.	5
Propósito del documento.	5
Alcance del sistema.	5
Definiciones relevantes para esta versión.	6
Actores del sistema para esta versión.	6
Actores internos.	6
Usuario Registrado.	6
Tecnologías Utilizadas.	7
Arquitectura General.	9
Patrón Arquitectónico Backend	9
Controller	9
Service	10
Repository	10
Manejo de Archivos	10
Manejo de correos electrónicos	10
Patrón Arquitectónico Frontend	11
Patrón Arquitectónico Frontend	11
Flujo de autenticación.	12
Inicio de sesión	12
Almacenamiento y uso del token	12
Protección de vistas	12
Cierre de sesión	12
Token inválido o inexistente	12
Requerimientos funcionales.	12
RF1: Registro de usuarios con credenciales locales	12
RF3: Autenticación de usuarios	13
RF4: Recuperación de contraseña	13
RF5: Cierre de sesión	13
RF6: Consulta y actualización de información del usuario autenticado:	13
RF7: Publicación de animales en adopción	13
RF8: Visualización de animales en mapa	13
RF9: Filtrado de animales	13
RF10: Consulta de ficha informativa de animal	13
RF11: Expresión de interés en adopción	13
RF12: Gestión de publicaciones propias	13
RF13: Inicio del proceso de adopción	13
RF14: Cierre del proceso de adopción	14
RF15: Reporte de publicaciones	14
RF16: Revisión de reportes	14
RF17: Baneo de usuarios	14

RF18: Aceptación de términos y condiciones	14
RF19: Control de acceso	14
RF20: Notificaciones a usuarios (Parcial)	14
RF21: Consulta de información de raza desde fuente externa	14
RF22: Registro de datos para compatibilidad entre adoptante y animal	14
Casos de uso.	14
De flujo de usuario	14
CU1: Registro de usuario local	14
CU3: Login	15
CU4: Recuperación de contraseña	15
CU5: Logout	16
Gestión de animales y publicaciones	17
CU6: Publicación de animal en adopción	17
CU7: Consulta y filtrado de animales en mapa	18
CU8: Dar "me interesa" a una publicación	18
CU9: Gestionar publicación de animal en adopción	19
CU10: Inicio de proceso de adopción	19
CU11: Generación de contrato	20
CU12: Cierre de proceso de adopción	20
Reportes, banos y revisiones	21
CU13: Reporte de publicación	21
CU14: Revisión de reporte por admin	21
CU15: Baneo de usuario	22
Perfil de usuario	23
CU16: Actualización de datos	23
CU17: Formulario de perfil de usuario	25
Requerimientos funcionales del MICROSERVICIO.	27
RF1: Procesamiento y Traducción LLM. El sistema debe recibir un nombre de raza y una	27
RF2: Mapeo de nivel de sociabilidad con mascotas, sociabilidad con niños, nivel de energía e independencia. El sistema debe asignar un valor numérico entero del 1 al 5 (donde 1 es muy bajo y 5 es muy alto) basado en el análisis del texto del temperamento ingresado.	27
RF3: Manejo de errores de validación (Cliente). Si el microservicio recibe peticiones con	27
RF4: Manejo de errores del Servicio LLM (Servidor). Si ocurre una falla en la	27
Casos de uso.	27
CU1: Clasificar temperamento de raza y traducir a español.	27
Modelo Conceptual de Datos	28
Requerimientos no funcionales.	29
RNF1: Seguridad de contraseñas	29
RNF2: Protección de funcionalidades con token de expiración de sesión	29

RNF3: Privacidad de usuarios y animales registrados	30
RNF4: Privacidad de la información de los usuarios	30
RNF5: Rendimiento de búsqueda y filtrado	30
RNF6: Procesamiento de imágenes rápido	30
RNF9: Registro rápido e intuitivo	30
RNF10: Sistema con soporte para múltiples usuarios concurrentes	30
Limitaciones Actuales	30
Funcionalidades pendientes para iteraciones futuras	30
Restricciones técnicas de la iteración	31

Introducción.

Propósito del documento.

El presente documento corresponde a la Iteración 4 del proyecto Adogta, desarrollado en el marco de la asignatura de Ingeniería de Software de la Facultad de Ciencias de la UNAM. Su propósito es evolucionar el documento entregado en la Iteración 3, reflejando el estado real del sistema actualmente implementado: un sistema completo que integra visualización en mapa, filtros avanzados, proceso completo de adopción (inicio y cierre), sistema de reportes de publicaciones, baneo de usuarios por parte del administrador, integración con The Dog API y The Cat API, y cuestionario de compatibilidad con algoritmo de recomendación K-Radius.

Se detallan todas las funcionalidades con las que el sistema cuenta actualmente, incluyendo los componentes incorporados en esta iteración. El documento sirve como manual descriptivo y base de conocimiento para nuevos desarrolladores.

Alcance del sistema.

Adogta es una plataforma web cuyo fin es facilitar el proceso de adopción de mascotas (perros y gatos). En esta **cuarta iteración**, el sistema expande su alcance para incluir:

- Registro de usuario con credenciales locales.
- Inicio y cierre de sesión.
- Recuperación de contraseña mediante correo electrónico.
- Cuestionario de compatibilidad adoptante-animal con algoritmo de recomendación.
- Publicación de animales en adopción.
- Gestión de publicaciones propias (edición y borrado lógico).
- **Visualización de animales en mapa** (ubicación aproximada por código postal).
- **Filtrado de animales** por tipo, raza, edad, código postal y compatibilidad.
- **Expresión de interés (“Me interesa”)** con notificación al donante.
- **Proceso de adopción** (inicio, generación de contrato en PDF, cierre).
- **Sistema de reportes** de publicaciones con revisión por administrador.
- **Baneo de usuarios** por parte del administrador.
- **Integración con The Dog API y The Cat API** para información de razas con un microservicio.
- **Notificaciones por correo** durante todo el flujo de adopción.

Las funcionalidades relativas a registro con Google OAuth y calificación de usuarios quedan para iteraciones futuras.

Definiciones relevantes para esta versión.

- **Usuario:** Cuenta registrada en la plataforma mediante credenciales como correo y contraseña. Se divide en Donante, Adoptante y Administrador. Es la entidad central del sistema.
- **Token de sesión:** Credencial generada al autenticarse exitosamente, utilizada para validar el acceso a rutas protegidas. Se almacena en sessionStorage en el frontend.
- **sessionStorage:** Mecanismo de almacenamiento del navegador utilizado para guardar el token de sesión. Los datos se eliminan al cerrar la pestaña o ventana del navegador.
- **Token de recuperación de contraseña:** Credencial temporal enviada por correo electrónico para permitir el restablecimiento de contraseña. Es de uso único y tiene tiempo de expiración definido.
- **Proveedor de autenticación:** Mecanismo utilizado para autenticar al usuario. En esta iteración únicamente está disponible el proveedor local (credenciales propias).
- **/me:** Endpoint de la API que devuelve la información del usuario que tiene sesión activa.
- **Vista protegida:** Página del frontend que requiere un token válido en sessionStorage para ser accedida. Si no existe token, el sistema redirige al login.
- **Formulario de compatibilidad:** Cuestionario que el adoptante completa una vez al año para registrar sus preferencias y condiciones de vida. El sistema utiliza esta información para determinar la compatibilidad con los animales disponibles.
- **Publicación:** Registro creado por un donante que contiene la información de un animal disponible para adopción, incluyendo nombre, tipo, raza, edad, fotos y descripción.
- **Solicitud ("Me interesa"):** Expresión de interés formal de un adoptante hacia una publicación específica. Queda registrada en el sistema y no puede duplicarse por adoptante y publicación.
- **Borrado lógico:** Mecanismo por el cual una publicación no se elimina físicamente de la base de datos sino que su estado cambia a "Borrada", dejando de ser visible para los usuarios.

Actores del sistema para esta versión.

Actores internos.

Usuario Registrado.

Persona que ha creado una cuenta en la plataforma proporcionando sus datos personales (nombre, apellidos, correo electrónico, contraseña, teléfono y código postal) y ha aceptado los términos y condiciones del sitio. En esta iteración el Usuario Registrado puede actuar como:

- **Donante:** cuando publica animales en adopción y gestiona sus propias publicaciones.

- **Adoptante:** cuando completa el formulario de compatibilidad y expresa interés en una publicación mediante "Me interesa".

Un mismo usuario registrado puede desempeñar ambos roles de forma simultánea.

Administrador

Usuario con rol especial (campo Admin en la BD). Sus capacidades incluyen:

- Ver panel de reportes.
- Revisar publicaciones reportadas (cambiar estado a “activa” o “baja por admin”).
- Aplicar baneos a usuarios infractores.

Tecnologías Utilizadas.

A continuación se describen las tecnologías empleadas en la implementación del backend y frontend de *Adogta* en la segunda iteración.

Backend

Categoría	Tecnología	Justificación
Lenguaje	Kotlin	Lenguaje moderno con tipado estático interoperabilidad con Java. Reduce la verbosidad del código y ofrece seguridad de nulos en tiempo de compilación
Framework	Spring Boot	Facilita la creación de APIs REST robustas con configuración mínima y soporte nativo para inyección de dependencias.
Base de Datos	PostgreSQL	Base de datos relacional de código abierto con soporte sólido para integridad referencial, transacciones y escalabilidad.
ORM	JPA / Hibernate	Permite mapear entidades kotlin a tablas de base de datos sin escribir SQL manual, agilizando el desarrollo y mantenimiento
Autenticación	JSON Web Tokens	Mecanismo sin estado para manejar sesiones de usuario. Permite validar identidad en cada petición sin consultar la base de datos.

Correo electrónico	SMTP / JavaMailSender	Utilizado para el envío de correos de recuperación de contraseña. Integrado mediante el servicio SmtplibService.
Archivos	Multipart / Sistema de archivos local	Las imágenes de los animales se reciben como archivos multipart, se almacenan en el sistema de archivos local bajo el directorio uploads/ y se sirven como recursos estáticos mediante el handler /uploads/**. El tamaño máximo por archivo y por petición es de 10 MB, configurado en WebMvcConfig.
Pruebas	Postman	Herramienta para verificar el comportamiento de extremo a extremo de los endpoints de la API
Control de versiones	Git / GitHub	Permite el trabajo colaborativo, historial de cambios y versionamiento formal mediante tags semánticas.

Microservicio

Lenguaje y Framework	Python 3 / FastAPI	Permite un desarrollo ágil de APIs asíncronas de alto rendimiento y proporciona documentación automática Swagger.
LLM / IA	LangChain y OpenAI	Se utilizan para orquestar los prompts y asegurar, con with_structured_output, que la respuesta siga estrictamente los esquemas definidos.
Validación de Datos	Pydantic	Garantiza que la estructura de los datos de entrada (texto, nombre) y salida (traducciones, nivel 1-5) sean consistentes antes de procesarlos o retornarlos.
Servidor Web	Uvicorn	Servidor ASGI ligero y ultrarrápido para ejecutar la aplicación FastAPI de manera local o en despliegue.

FrontEnd

Categoría	Tecnología	Justificación
Framework	React 18	Biblioteca de JavaScript para construir interfaces de usuario mediante componentes reutilizables. Ofrece un modelo declarativo que facilita la gestión del estado y la actualización de la interfaz.
Builder	Vite	Herramienta de desarrollo moderna que ofrece inicio instantáneo del servidor y compilación optimizada.

		Sustituye a Create React App con mejor rendimiento.
Estilos	Tailwind css	Framework de CSS utilitario que permite construir interfaces directamente en el HTML sin escribir hojas de estilo separadas. Agiliza el desarrollo y garantiza consistencia visual.
Comunicación con API	Axios	Biblioteca HTTP para realizar peticiones al backend. Permite configurar interceptores para adjuntar el token de autorización automáticamente en todas las peticiones protegidas.
Gestión de sesión	sessionStorage	Mecanismo nativo del navegador para almacenar el token de sesión. Los datos persisten durante la sesión activa y se eliminan al cerrar la pestaña.
Control de versiones	Git / GitHub	Repositorio independiente del backend. El frontend se versiona con el tag 2.0.0 en esta iteración.

Para el mapa

API de Mapas	Google Maps JavaScript API	Proporciona mapas interactivos, marcadores personalizables y ventanas de información (InfoWindow). Se integra mediante la librería @react-google-maps/api en el frontend.
Geocodificación	Tabla estática de coordenadas (PostgreSQL) (Kotlin/Spring Boot)	Convierte código postal en coordenadas (lat/lng) sin depender de APIs externas de geocodificación, evitando costos y latencias. Se utiliza una tabla precargada con los códigos postales de México.
Visualización de pines	Marker estándar + MarkerClusterer (Google Maps library)	Permite mostrar pines individuales para un animal por CP
Frontend – Mapa	@react-google-maps/api (React wrapper)	Simplifica la carga y el manejo del mapa en React, gestiona el ciclo de vida del script de Google Maps y expone hooks para interactuar con el mapa y los marcadores.
Filtros en tiempo real	React (useState, useEffect) + Axios	Los filtros (tipo de animal, raza, código postal) modifican el estado local del frontend; se vuelve a consultar el backend o se filtra la lista de publicaciones, y los marcadores se actualizan dinámicamente sin recargar la página.

Arquitectura General.

El sistema Adogta está compuesto por dos componentes independientes que se comunican entre sí mediante HTTP:

- **Frontend:** interfaz de usuario desarrollada en React que consume la API REST del backend. Se ejecuta en el navegador del usuario y gestiona el flujo de navegación, la presentación de datos y la autenticación del lado del cliente.
- **Backend:** API REST organizada en capas (Controller, Service, Repository) que expone los endpoints del sistema, gestiona la lógica de negocio y persiste los datos en PostgreSQL.
- **Comunicación:** el frontend realiza peticiones HTTP al backend mediante Axios. En las peticiones que requieren autenticación, el token almacenado en sessionStorage se envía en el encabezado Authorization. El backend tiene configurado CORS para aceptar peticiones desde `http://localhost:5173`, que es el origen del servidor de desarrollo de Vite.

Patrón Arquitectónico Backend

La arquitectura sigue el patrón MVC adaptado a REST, donde cada capa tiene una responsabilidad específica y bien delimitada:

Controller

Es el punto de entrada de las peticiones HTTP. Recibe las solicitudes del cliente, valida la presencia del token cuando el endpoint es protegido, y delega la lógica de negocio al Service. Retorna las respuestas en formato JSON con los códigos HTTP apropiados. No contiene lógica de negocio. En esta iteración se incorporaron cuatro nuevos controladores: `AnimalController`, `FormularioController`, `SolicitudController` y `RazaController`.

Service

Contiene toda la lógica de negocio del sistema. Es responsable de orquestar las operaciones, aplicar validaciones de negocio, manejar excepciones y coordinar las interacciones entre múltiples repositorios cuando es necesario. No accede directamente a la base de datos. Destacan en esta iteración `AnimalService`, que gestiona la publicación, edición, listado y borrado lógico de animales de forma transaccional, y `FormularioService`, que valida el token del usuario, persiste las respuestas del cuestionario y actualiza el estado del cuestionario del usuario en la base de datos.

Repository

Es la capa de acceso a datos. Se encarga de la comunicación con la base de datos mediante operaciones CRUD. Abstrae los detalles de persistencia del resto de la aplicación. En esta iteración se incorporaron `AnimalRepository`, `PublicacionRepository`, `FormularioRepository`, `RazaRepository` y `SolicitudRepository`.

Manejo de Archivos

Las imágenes de los animales se reciben como archivos multipart a través de UploadController, se almacenan en el sistema de archivos local bajo el directorio uploads/ y se sirven como recursos estáticos mediante el handler /uploads/**. El tamaño máximo permitido es de 10 MB por archivo y por petición, configurado en WebMvcConfig.

Manejo de correos electrónicos

El sistema implementa el envío de correos mediante una arquitectura de dos capas: una interfaz EmailService que define el contrato de las operaciones de notificación, y una implementación concreta SmtplibEmailService que contiene la lógica real de envío. Este diseño desacopla el resto del sistema de la implementación específica de correo, lo que permite cambiar el proveedor en el futuro sin modificar los servicios que lo invocan.

La implementación utiliza JavaMailSender de Spring Boot Mail, que se configura automáticamente mediante las propiedades spring.mail.* del archivo application.properties. El servidor de correo utilizado es SMTP. La URL base de la aplicación se inyecta mediante la propiedad app.base-url, lo que permite que los enlaces generados en los correos funcionen correctamente tanto en desarrollo como en un eventual entorno de producción sin modificar el código.

El contenido visual de los correos se genera mediante **Thymeleaf**, un motor de plantillas que procesa archivos HTML antes de enviarlos. El sistema cuenta con dos plantillas:

- email-recuperacion: utilizada en el flujo de recuperación de contraseña. Recibe como variable el enlace completo con el token temporal (\$baseUrl/reset-password?token={token}) para que el usuario pueda restablecer su contraseña directamente desde el correo.
- email-solicitud-adopcion: utilizada cuando un adoptante expresa interés en una publicación. Recibe el nombre del animal, el nombre completo del adoptante, su correo electrónico y su teléfono (opcional), y notifica al donante con los datos de contacto necesarios para continuar el proceso de adopción.

Todos los correos se envían en formato HTML con codificación UTF-8. Si el servidor SMTP no está disponible o el envío falla por cualquier razón, Spring lanza una MailSendException que es manejada por el service correspondiente.

Patrón Arquitectónico del MICROSERVICIO

La arquitectura del microservicio sigue un modelo por capas o módulos bien delimitados para garantizar la escalabilidad y facilitar pruebas:

- Routes (app/api/routes.py): Es el punto de entrada. Define el endpoint POST /clasificar-temperamento. Recibe la petición HTTP y delega la ejecución al servicio.
- Service (app/services/llm_service.py): Contiene toda la lógica de negocio y la conexión con el modelo de OpenAI. Utiliza un ChatPromptTemplate para inyectar el contexto de experto en comportamiento canino.
- Models/Schemas (app/models/schemas.py): Define los esquemas ClasificacionRequest (nombre, texto_a_clasificar) y ResultadoClasificacion (nombre en español, temperamento en español, nivelEnergia).
- Configuración (app/core/config.py): Se encarga de cargar y validar las variables de entorno, aislando credenciales sensibles (API Key) del resto del código.

Patrón Arquitectónico Frontend

El frontend se organiza en módulos con responsabilidades separadas:

- pages/: contiene las vistas principales. En esta iteración se incorporaron PublicarAnimalPage, EditarAnimalPage, QuestionnairePage, ForgotPasswordPage y ResetPasswordPage.
- components/common/: componentes reutilizables de interfaz (Button, Input, Checkbox, LoadingSpinner).
- components/profile/: componentes específicos del perfil de usuario. Se incorporó QuestionnaireForm, que renderiza el formulario de compatibilidad.
- hooks/: lógica reutilizable encapsulada en custom hooks (useAuth, useForm, useLoading).
- api/: capa de comunicación con el backend. En esta iteración se incorporaron animalApi.js, formularioApi.js, razaApi.js, solicitudApi.js y uploadApi.js.
- layouts/: estructuras de página reutilizables (AuthLayout, PublicLayout, Header, Footer).
- modals/: componentes de ventana modal (EditProfile, Terms).
- utils/: funciones utilitarias. Se incorporó questionnaire.js con la lógica de procesamiento del cuestionario de compatibilidad.

Flujo de autenticación.

Esta sección describe conceptualmente cómo el sistema gestiona la autenticación del usuario de extremo a extremo, desde la interfaz hasta el backend.

Inicio de sesión

El usuario accede a la vista de Login e ingresa su correo electrónico y contraseña. Al enviar el formulario, el frontend realiza una petición al endpoint de autenticación del backend. Si las credenciales son correctas, el backend genera y devuelve un token de sesión. El frontend almacena ese token en sessionStorage y redirige al usuario al Dashboard.

Almacenamiento y uso del token

Una vez almacenado el token, Axios lo incluye automáticamente en el encabezado Authorization de todas las peticiones posteriores que requieran autenticación. De esta forma, el backend puede verificar la identidad del usuario en cada petición sin que el usuario deba ingresar sus credenciales nuevamente.

Protección de vistas

Las vistas que requieren autenticación (Dashboard, Perfil, Actualización de perfil) verifican al cargarse si existe un token válido en sessionStorage. Si no existe o ha expirado, el sistema redirige automáticamente al usuario a la vista de Login sin mostrar el contenido protegido.

Cierre de sesión

Al cerrar sesión, el frontend envía una petición al endpoint de logout del backend, que invalida el token en el servidor. El frontend elimina el token de sessionStorage y redirige al usuario al Login. A partir de ese momento, el usuario queda sin sesión activa.

Token inválido o inexistente

Si el backend recibe una petición con un token inválido, expirado o ausente, responde con HTTP 401. El frontend interpreta esta respuesta, elimina cualquier token almacenado en sessionStorage y redirige al usuario al Login con un mensaje de error visible.

Requerimientos funcionales.

Los siguientes requerimientos funcionales corresponden a las funcionalidades implementadas en la Iteración 4, relacionadas con la entidad Usuario en el sistema completo (frontend + backend).

RF1: Registro de usuarios con credenciales locales

El sistema permitirá a los usuarios registrarse proporcionando datos personales: nombre(s), apellido paterno, apellido materno, correo electrónico, contraseña, código postal y número de teléfono. La contraseña deberá tener al menos 8 caracteres. El correo electrónico deberá ser único en el sistema. El frontend expone un formulario de registro que valida los campos antes de enviar la petición al backend.

RF3: Autenticación de usuarios

El sistema permitirá que los usuarios verifiquen sus credenciales (correo electrónico y contraseña) para acceder a la plataforma. Al autenticarse exitosamente, el backend genera un token de sesión que el frontend almacena en sessionStorage y utiliza en peticiones subsecuentes a rutas protegidas.

RF4: Recuperación de contraseña

Se le permitirá a los usuarios restablecer su contraseña mediante un correo electrónico de recuperación.

RF5: Cierre de sesión

El usuario puede cerrar su sesión desde la interfaz. El frontend envía la solicitud de logout al backend, que invalida el token. El frontend elimina el token de sessionStorage y redirige al usuario al Login.

RF6: Consulta y actualización de información del usuario autenticado:

El sistema expone el endpoint /me que retorna los datos del perfil del usuario con sesión activa. Adicionalmente, en esta iteración se implementa el endpoint de actualización (PUT /usuarios) que permite modificar los datos del usuario autenticado. El frontend consume ambos endpoints a través de las vistas de Perfil y Actualización de perfil.

RF7: Publicación de animales en adopción

Se le permite a un usuario donante crear publicaciones para dar detalles de una mascota (fotos, raza, edad, vacunas, etc) para ponerla en adopción.

RF8: Visualización de animales en mapa

Se mostrará en un mapa los animales disponibles para adopción según el código postal registrado. No se revelarán ubicaciones exactas.

RF9: Filtrado de animales

La plataforma le permitirá a los usuarios filtrar los animales disponibles por datos como tipo, raza, código postal, edad, etc.

RF10: Consulta de ficha informativa de animal

Se mostrará una ficha informativa con los detalles específicos de cada animal en las publicaciones.

RF11: Expresión de interés en adopción

Se le permitirá a los usuarios adoptantes hacer click en un botón “Me interesa” de una publicación para iniciar el proceso de adopción del animal.

RF12: Gestión de publicaciones propias

Los usuarios donantes tendrán permitido verificar, modificar y marcar como adoptados los animales que han publicado en la plataforma.

RF13: Inicio del proceso de adopción

El sistema le permitirá al donante iniciar el proceso de adopción de un animal con un adoptante.

RF14: Cierre del proceso de adopción

Se le permitirá a los usuarios marcar como adoptado al animal una vez concluido el proceso de adopción tras el contacto y acuerdo entre donante y adoptante. Se consultará a ambas partes para verificar que el proceso ha concluido.

RF15: Reporte de publicaciones

La plataforma le permitirá a los usuarios registrados reportar publicaciones que consideren sospechosas o inapropiadas. Se deberá registrar un motivo de reporte.

RF16: Revisión de reportes

Se le proporcionará a un usuario administrador un panel para controlar los reportes registrados. Se deja a criterio del administrador las medidas a tomar.

RF17: Baneo de usuarios

Un usuario administrador puede suspender la cuenta de un usuario reportado que haya infringido las normas de la plataforma.

RF18: Aceptación de términos y condiciones

Se requiere que los usuarios acepten los términos y condiciones de la plataforma antes o después (por potencial actualización de los términos) de su registro.

RF19: Control de acceso

El sistema restringirá el acceso a las rutas protegidas tanto en backend como en frontend verificando la validez del token de sesión en cada petición. Las rutas que requieren autenticación retornarán HTTP 401 si el token es inválido o ha expirado.

RF20: Notificaciones a usuarios (Parcial)

El sistema enviará correos a los usuarios en eventos concretos como cambio de contraseña o novedades sobre el proceso de adopción.

RF21: Consulta de información de raza desde fuente externa

Se integrará el sistema con las APIs *The Dog API* y *The Cat API* para mostrar detalles más específicos sobre las razas de los animales en las fichas descriptivas.

RF22: Registro de datos para compatibilidad entre adoptante y animal

Se le permite a los usuarios brindar detalles personales mediante un formulario para verificar que el animal de interés sea compatible con el potencial adoptante.

Casos de uso.

De flujo de usuario

CU1: Registro de usuario local

Actores: Persona interesada en usar la aplicación

Descripción: Una persona interesada en la aplicación desea registrarse en la misma, por lo que el sistema debe manejar este registro de manera exitosa; un registro se define como exitoso cuando los datos de dicho usuario quedan guardados en el sistema y ahora es un usuario.

Precondiciones:

- La persona cuenta con un correo electrónico válido.

Flujo principal:

1. La persona se encuentra en la sección de registro.
2. La persona ingresa su correo electrónico en la sección correspondiente.
3. A la persona se le pide crear una contraseña.
4. Se guarda a la persona en la base de datos.

Flujo alternativo:

- Si la persona no ingresa un correo electrónico válido el sistema rechaza el registro.
- Si la persona no crea una contraseña de al menos 8 caracteres, se rechaza.

Postcondición:

- La persona queda registrada dentro del sistema, por lo que ahora es un usuario de la aplicación y puede iniciar sesión e interactuar con las demás funcionalidades del sistema.

CU3: Login

Actores: Usuario de la aplicación.

Descripción: Un usuario desea iniciar sesión en la aplicación, por lo que este inicio de sesión debe ser manejado de manera correcta, lo que se traduce en que el usuario accede a la aplicación y puede interactuar con la misma.

Precondiciones:

- El usuario tiene una cuenta activa en la aplicación.

Flujo principal:

1. El usuario se encuentra en la sección de login.
2. El usuario inicia sesión con su correo electrónico y contraseña o con su cuenta de Google.
3. Se valida que el usuario que intenta iniciar sesión esté registrado y con una cuenta activa.
4. Se le permite ingresar al usuario.

Flujo alternativo:

- Si el usuario cuenta con una cuenta activa, se rechaza el inicio de sesión.
- Si la contraseña del usuario es incorrecta, se rechaza el inicio de sesión.
- Si la cuenta de Google con la que se intenta iniciar sesión no se encuentra registrada en el sistema, se rechaza dicho inicio de sesión.

Postcondición:

- El usuario inicia sesión exitosamente y puede interactuar con la aplicación.

CU4: Recuperación de contraseña

Actores: Usuario de la aplicación.

Descripción: Un usuario de la aplicación extravió su contraseña, por lo que se le ofrece la opción de restablecer, las instrucciones son dadas mediante un correo electrónico y este cambio de contraseña debe verse reflejado en el sistema, es decir, la nueva contraseña debe ser la nueva y única contraseña con la que el usuario puede iniciar sesión.

Precondiciones:

- El usuario tiene una cuenta activa en la aplicación.

Flujo principal:

1. El usuario se encuentra en la sección de login.
2. El usuario da click en la opción “Olvidé mi contraseña”
3. Se envía un correo electrónico al usuario con las instrucciones para restablecer la contraseña.
4. Dentro de la base de datos, se actualiza la contraseña del usuario.

Flujo alternativo:

- Si el usuario no tiene una cuenta activa en la aplicación no recibirá ningún correo,
- Si la nueva contraseña del usuario no cumple con las condiciones, se rechaza.

Postcondición:

- El usuario puede iniciar sesión de manera exitosa con la nueva contraseña, y por lo tanto, puede interactuar con la aplicación.

CU5: Logout

Actores: Usuario de la aplicación.

Descripción:

Un usuario con sesión activa elige cerrar sesión. El sistema invalida el token de sesión en el servidor, garantizando que ninguna petición futura con ese token sea aceptada.

Precondiciones:

- El actor ha iniciado sesión previamente y posee un token de sesión válido.
- El token de sesión existe en el servidor y no ha expirado.

Flujo principal:

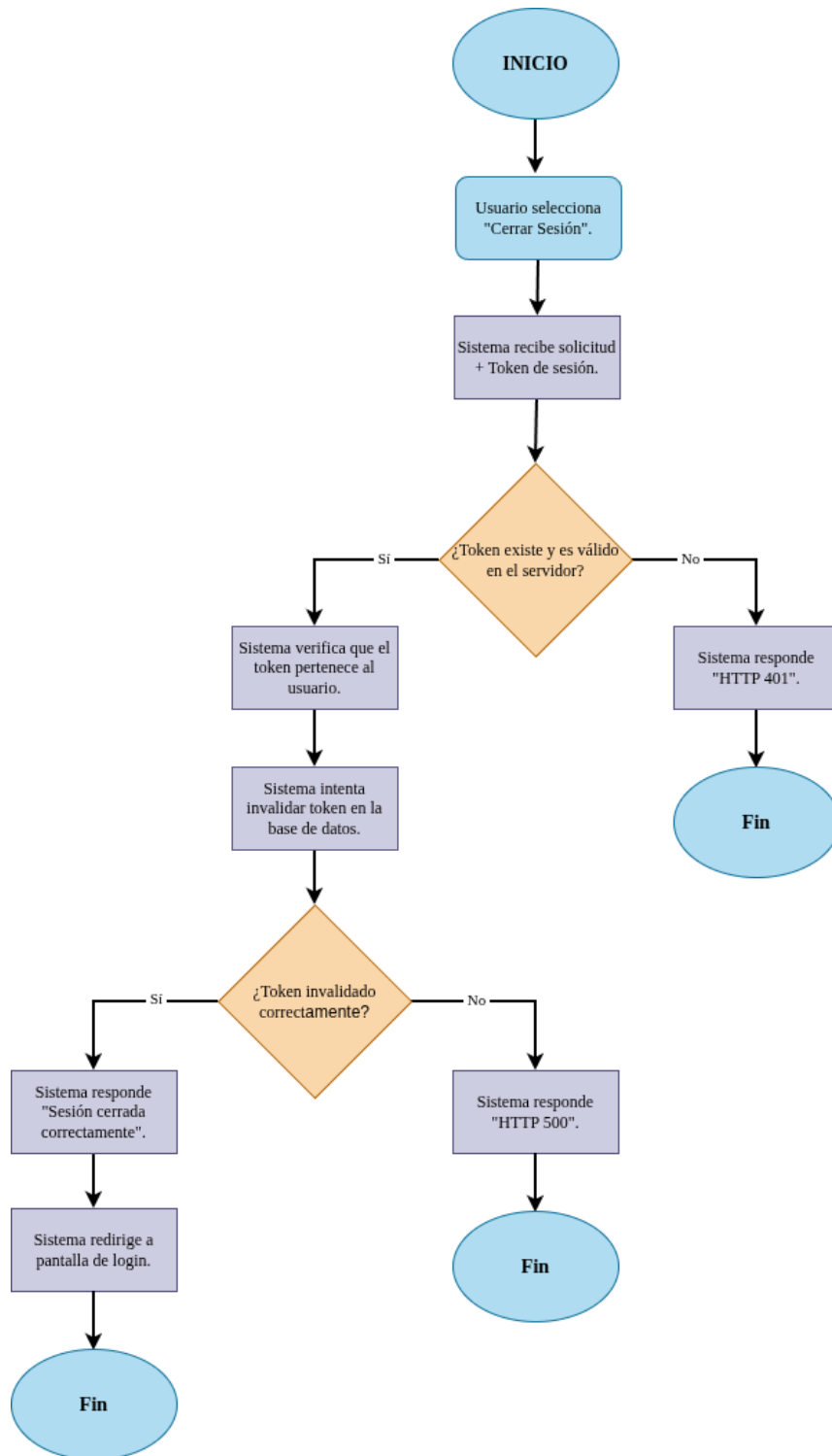
1. El usuario selecciona la opción “Cerrar sesión” en la interfaz.
2. El sistema recibe la solicitud de logout junto con el token de sesión activo.
3. El sistema verifica que el token existe y está asociado al usuario.
4. El sistema invalida el token en la base de datos (lo elimina o marca como inválido).
5. El sistema responde con una confirmación de cierre de sesión exitoso.
6. El sistema redirige al usuario a la página de inicio de sesión.

Flujo alternativo:

- Flujo alternativo 1. Token ya invalido o expirado: el sistema detecta que el token no existe o expiró; responde con un código HTTP 401.
- Flujo alternativo 2. Error del servidor: si la invalidación falla, el sistema registra el error internamente y responde con HTTP 500 sin exponer detalles al usuario.

Postcondiciones:

- El token de sesión queda invalidado en el servidor y no puede reutilizarse.
- El usuario no puede acceder a funcionalidades protegidas hasta iniciar sesión nuevamente.



Gestión de animales y publicaciones

CU6: Publicación de animal en adopción

Actores: Donante

Descripción:

Permite al usuario dar de alta a su perro/gato en la plataforma para que lo puedan ver las demás personas.

Precondiciones:

- Que el usuario haya iniciado sesión

Flujo principal:

1. El usuario le da click en el botón de “Registrar animal”
2. El sistema te solicitará un tipo de cuestionario donde te pedirá nombre, descripción, fotos, raza, etc...
3. El sistema igual checará con la API si se puede sugerir características de la raza.
4. El usuario debe poner al menos una foto.
5. El sistema revisará si todos los campos se completaron correctamente.
6. El sistema guardará el registro y también generará un tipo marcador en el mapa, esto en base al CP.

Flujos alternativos:

- FA1. Si falta info obligatoria o si la foto no cumple el formato indicado, el sistema mostrará un error y no guardarán cambios.

Postcondición:

- El perro/gato registrado es visible en la galería y en el mapa.

CU7: Consulta y filtrado de animales en mapa

Actores: Donante

Descripción:

Permite al usuario ver la ubicación aproximada de las mascotas disponibles y puede aplicar filtros también.

Precondiciones:

- Que el usuario haya iniciado sesión

Flujo principal:

1. El usuario entrará a la sección del mapa de adopción.
2. El sistema cargará los marcadores/pines disponibles usando la API de Google Maps.
3. El usuario filtrará a través de los criterios disponibles.
4. El sistema actualizará el mapa junto a los marcadores para que se cumpla con el filtrado.
5. El usuario le da click en algún marcador para ver la vista previa del perro/gato.

Flujos alternativos:

- FA1. Si no hay animales que cumplan con los criterios de filtrado del usuario, el sistema lanzará un error diciendo que no hay resultados.

Postcondición:

- El sistema arroja los resultados filtrados visualmente y en el mapa.

CU8: Dar "me interesa" a una publicación

Actores: Donante

Descripción:

Permite al usuario mostrar interés a la mascota para contactarse con el donante.

Precondiciones:

- Que el usuario haya iniciado sesión
- Que el usuario haya completado el formulario de compatibilidad.

Flujo principal:

1. El usuario puede ver la información de la mascota.
2. El usuario le da click en “Me interesa”.
3. El sistema registra al adoptante interesado y la publicación.
4. El sistema le manda una notificación al donante informando sobre el nuevo interesado.

Flujos alternativos:

- FA1. Si el usuario no cumple con el perfil de compatibilidad, el sistema lo redirigirá al formulario.

Postcondición:

- El interés queda registrado en el sistema, y el donante puede ver el perfil del interesado.

CU9: Gestionar publicación de animal en adopción

Actores: Donante

Descripción:

Permite al usuario que publicó una mascota modificar sus datos o eliminar la publicación.

Precondiciones:

- Que el usuario haya iniciado sesión y que sea autor de la publicación.

Flujo principal:

1. El usuario accede a “Mis publicaciones”.
2. El sistema muestra la lista de las mascotas registradas por el usuario.
3. El usuario selecciona “Editar” o “Eliminar” en una publicación específica.
4. Si selecciona “Editar”: sólo modifica los campos obligatorios, el sistema lo checa y guarda los cambios.
5. Si selecciona “Eliminar”: sólo se solicita la confirmación, y se hace el borrado lógico.

Postcondición:

- La información de la publicación se actualiza o deja de estar visible en la plataforma.

CU10: Inicio de proceso de adopción

Actores: Donante

Descripción:

El donante elige a un adoptante interesado para comenzar formalmente los trámites de adopción.

Precondiciones:

- Que el usuario haya iniciado sesión y que exista uno o más “Me interesa” en la publicación.

Flujo principal:

1. El donante checa la lista de los interesados en su publicación.
2. El donante selecciona a un interesado en específico basándose en su perfil de compatibilidad.
3. El donante hace click en “Iniciar proceso de adopción”.

4. El sistema cambia el estado de la publicación a “En proceso”.
5. El sistema envía correo tanto al adoptante como al donante, con la información de contacto y los pasos a seguir.

Postcondición:

- La publicación se bloquea para nuevos interesados mientras el proceso esté activo.

CU11: Generación de contrato

Actores: Sistema, Donante y Adoptante.

Descripción:

El sistema genera un documento con los datos del donante y adoptante y las responsabilidades de la adopción.

Precondiciones:

- El proceso de adopción ya inició.

Flujo principal:

1. El sistema junta toda la información de los perfiles de ambos usuarios.
2. El sistema genera un PDF con los términos y condiciones de adopción aceptados anteriormente.
3. El sistema envía por correo a los dos, para su firma digital o física.

Postcondición:

- Hay un documento legal que respalda la adopción de la mascota.

CU12: Cierre de proceso de adopción

Actores: Donante, Adoptante

Descripción:

Se formaliza la entrega de la mascota y se finaliza la publicación en el sistema.

Precondiciones:

- Que el proceso de adopción esté en su última etapa.

Flujo principal:

1. El donante marca la publicación como “Adoptado”.
2. El sistema envía una solicitud de confirmación al adoptante.
3. El adoptante también confirma la adopción exitosa.
4. El sistema archiva la publicación.

Flujos alternativos:

- FA1. Si alguna de las partes cancela el proceso, el sistema les pide el motivo y vuelve a poner la publicación como "Activa" para otros interesados.

Postcondición:

- La mascota sale como adoptada y el ciclo de vida de la publicación termina exitosamente.

Reportes, baneos y revisiones

CU13: Reporte de publicación

Actores: Usuario registrado, Administrador

Descripción:

Un usuario reporta una publicación por contenido inapropiado, spam, información falsa u otros motivos. Múltiples reportes pueden escalar a revisión por parte de un administrador.

Precondiciones:

- El usuario ha iniciado sesión
- La publicación reportada está visible (estado “activa”)
- El usuario no es el propietario de la publicación

Flujo principal:

1. El usuario accede a la ficha de una publicación y selecciona “Reportar”
2. El usuario selecciona el motivo del reporte (foto inapropiada, spam, publicación falsa, información falsa) y puede agregar un comentario opcional.
3. El sistema registra el reporte con: id_reporte, motivo, fecha, estado = “pendiente” y la referencia a la publicación y al usuario que reportó
4. El sistema verifica la cantidad de reportes activos para esa publicación
5. Si el número de reportes supera el umbral definido, el sistema cambia automáticamente el estado de la publicación a “en revisión” y notifica al administrador.
6. El sistema confirma al usuario que el reporte fue recibido

Flujos alternativos:

- FA1: El usuario ya reportó esa publicación anteriormente: el sistema informa que ya existe un reporte de este usuario y no registra un duplicado
- FA2: El administrador no está disponible para revisar en tiempo real: el reporte queda en cola con estado pendiente hasta ser atendido

Postcondición:

- Existe un reporte registrado vinculado a la publicación
- Si se superó el umbral, la publicación está en estado de revisión
- El administrador es notificado si aplica

CU14: Revisión de reporte por admin

Actores: Administrador

Descripción:

El Administrador revisa una publicación que fue marcada como "en revisión" por acumulación de reportes y decide su destino.

Precondiciones:

- Existe al menos una publicación en estado “en revisión”
- El administrador ha iniciado sesión

Flujo principal:

1. El administrador accede al panel de reportes
2. El administrador selecciona una publicación en revisión y consulta los reportes acumulados con sus motivos.
3. El administrador revisa el contenido de la publicación (fotos, descripción, información del animal)
4. Pasa una de 2 cosas: se desestima el reporte, la publicación regresa y se pone en estado "activa" y los reportes se marcan como "desestimado". Se notifica al donante que su publicación fue realizada y está activa. También puede suceder que se de baja a la publicación y el estado pasa a "baja por admin" y se notifica al donante informando el motivo
5. El administrador actualiza el estado de los reportes sueltos

Flujos alternativos

- FA1: El administrador determina que el usuario tiene un patrón de comportamiento abusivo. Para verificar esto solo se necesita el patrón de comportamiento abusivo.

Postcondición:

- La publicación tiene un estado definitivo
- Los reportes asociados están en estado "resuelto"
- El donante fue notificable

CU15: Baneo de usuario

Actores: Administrador

Descripción:

El Administrador aplica un ban a un usuario, impidiendo que acceda a la plataforma.

Precondiciones:

- El administrador ha iniciado sesión
- Existe evidencia de comportamiento que justifica el ban (reportes acumulados, revisión manual.)

Flujo principal:

1. El Administrador accede al perfil del usuario infractor.
2. El Administrador selecciona "Aplicar ban".
3. El Administrador ingresa el motivo del ban.
4. El sistema crea un registro de Ban con: id_ban, motivo, fecha, referencia al Administrador que lo aplicó y al usuario baneado.
5. El sistema valida el token de sesión activo del usuario baneado.
6. El sistema marca las publicaciones activas del usuario como "suspendidas por ban".
7. El sistema notifica al usuario por correo que su cuenta fue suspendida y el motivo.v

Flujos alternativos

- FA1: El usuario ya tiene un ban activo: el sistema informa al Administrador y le permite actualizar el motivo o registrar un ban adicional.
- El Administrador decide aplicar un ban temporal: el sistema registra una fecha de expiración del ban.

Postcondición:

- Existe un registro de Ban asociado al usuario.

- El usuario no puede iniciar sesión; si lo intenta, recibe el mensaje de cuenta suspendida (FA2 de CU3).
- Sus publicaciones activas fueron suspendidas.

Perfil de usuario

CU16: Actualización de datos

Actores: Usuario autenticado.

Descripción:

El usuario autenticado modifica uno o más campos de su perfil; nombres, apellidos, correo electrónico, número de teléfono, código postal o contraseña. Si el usuario cambia el correo, el sistema envía un mensaje de verificación al nuevo correo antes de aplicar el cambio.

Precondiciones:

- El usuario inició sesión y posee un token de sesión válido.
- El usuario ha accedido a la sección de perfil o configuración.

Flujo principal:

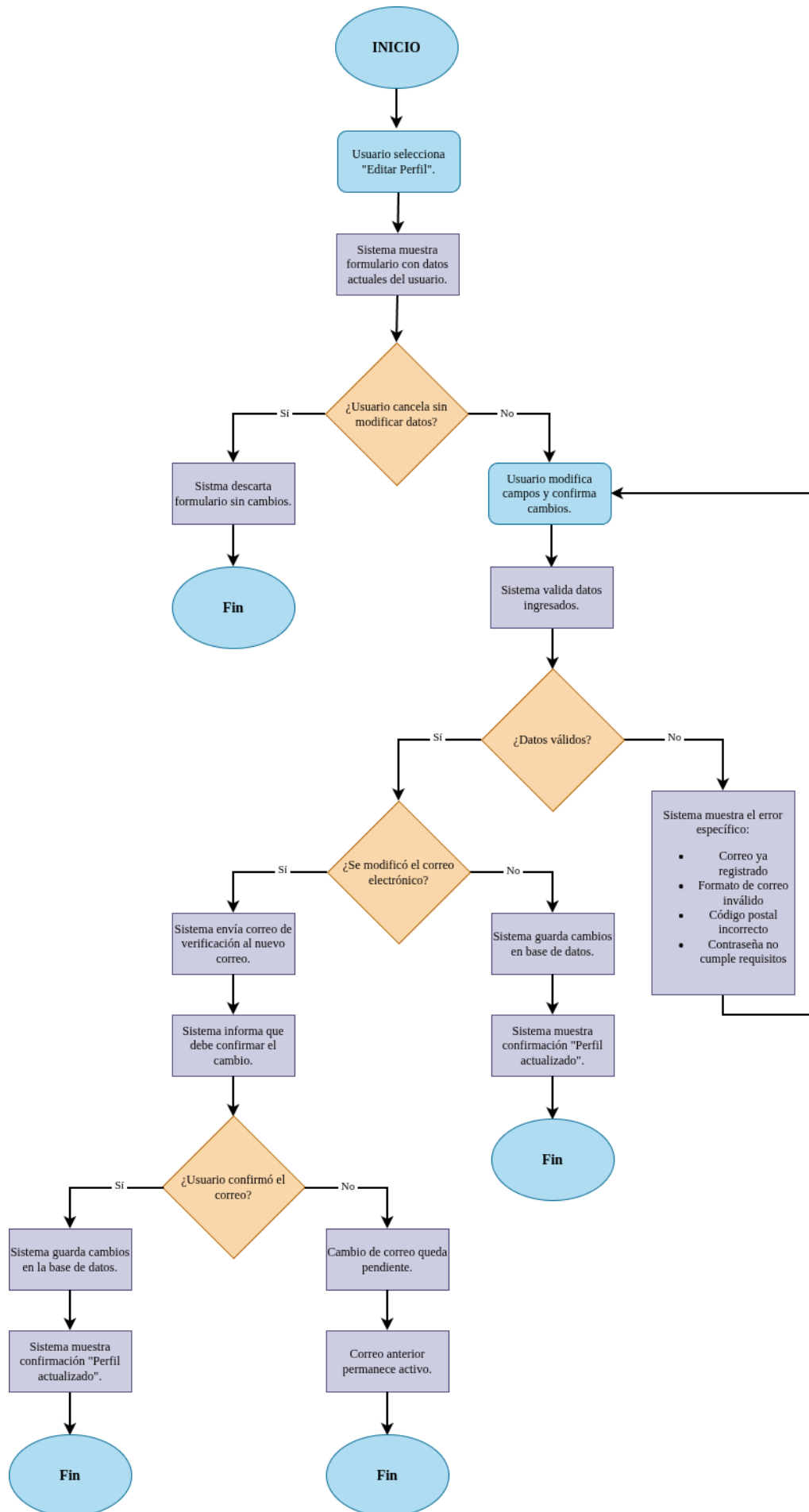
1. El actor accede a la sección “Editar perfil”.
2. El sistema muestra el formulario con los datos actuales del usuario.
3. El actor modifica los campos deseados y confirma los cambios.
4. El sistema valida los datos ingresados (formato, completitud, unicidad del correo).
5. Si el correo fue modificado el sistema envía un correo de verificación al nuevo correo e informa al usuario que debe confirmar el cambio.
6. Si el correo no fue modificado el sistema aplica los cambios directamente.
7. El sistema guarda los datos actualizados y muestra confirmación de actualización exitosa.

Flujos alternativos:

- Flujo alternativo 1. Nuevo correo ya registrado: el sistema muestra error indicando que el correo ya está en uso y no aplica el cambio.
- Flujo alternativo 2. Formato de correo inválido: el sistema señala el campo y solicita corregirlo antes de guardar.
- Flujo alternativo 3. Código postal con formato incorrecto: el sistema indica error y no guarda los cambios hasta corregirlo.
- Flujo alternativo 4. Contraseña nueva no cumple requisitos mínimos: el sistema muestra los requisitos y no aplica el cambio.
- Flujo alternativo 5. El actor no confirma el correo de verificación: el correo anterior permanece activo; el cambio queda pendiente hasta completar la verificación.
- Flujo alternativo 6. El actor no realiza ningún cambio y cancela: el sistema descarta el formulario sin modificar datos.

Postcondición:

- Los campos modificados quedan actualizados en la base de datos.
- Si se cambió el correo y fue verificado, el nuevo correo es el correo activo del usuario.



CU17: Formulario de perfil de usuario

Actores: Usuario autenticado (recién registrado o perfil incompleto).

Descripción:

Tras completar el registro, el sistema detecta que el perfil de usuario está incompleto y presenta un formulario para que el actor proporcione la información complementaria: apellidos, número de teléfono y aceptación de términos.

Precondiciones:

- El usuario completó el flujo de registro (CU1.1 CU1.2) o inicia sesión.
- El perfil del usuario no contiene todos los campos complementarios.

Flujo principal:

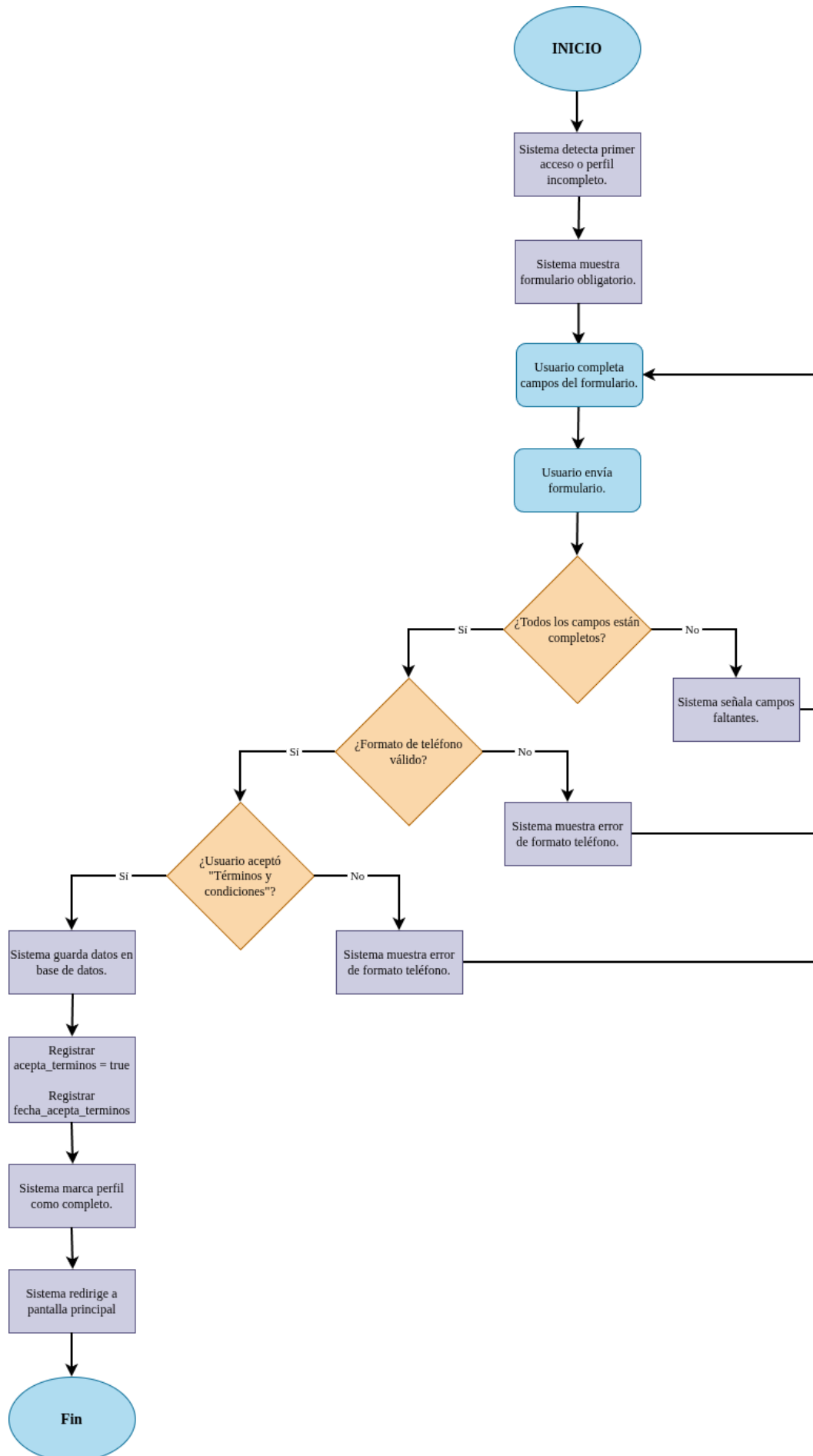
1. El sistema detecta que es el primer acceso del usuario o que el perfil está incompleto.
2. El sistema muestra el formulario obligatorio: apellido paterno, apellido materno, teléfono y aceptación de términos y condiciones.
3. El usuario completa todos los campos y acepta los términos.
4. El sistema valida formato de teléfono y que todos los campos estén llenos.
5. El sistema guarda los datos y registra `acepta_terminos=true` y `fecha_acepta_terminos`.
6. El sistema redirige al actor a la pantalla principal con acceso completo.

Flujos alternativos:

- Flujo alternativo 1. Campo vacío o incompleto: el sistema señala los campos faltantes y no permite avanzar hasta completarlos todos.
- Flujo alternativo 2. Formato de teléfono inválido: el sistema indica el error y bloquea el avance.
- Flujo alternativo 3. Términos no aceptados: el sistema no permite continuar hasta que el usuario los acepte.

Postcondición:

- El perfil del usuario queda marcado como completo en el sistema.
- Los campos `apellido_paterno`, `apellido_materno`, `teléfono` quedan almacenados.
- `acepta_terminos` queda marcado como verdadero y `fecha_acepta_terminos` queda registrada.
- El actor tiene acceso completo a las funcionalidades correspondientes al usuario.



Requerimientos funcionales del MICROSERVICIO.

Los siguientes requerimientos funcionales corresponden a las funcionalidades implementadas en la Iteración 4.

RF1: Procesamiento y Traducción LLM. El sistema debe recibir un nombre de raza y una descripción de temperamento en formato JSON, procesarlos utilizando un modelo LLM y retornar dichos valores traducidos íntegramente al español.

RF2: Mapeo de nivel de sociabilidad con mascotas, sociabilidad con niños, nivel de energía e independencia. El sistema debe asignar un valor numérico entero del 1 al 5 (donde 1 es muy bajo y 5 es muy alto) basado en el análisis del texto del temperamento ingresado.

RF3: Manejo de errores de validación (Cliente). Si el microservicio recibe peticiones con formato inválido, campos vacíos o tipos de datos incorrectos, debe retornar de forma inmediata un error HTTP 500 al backend.

RF4: Manejo de errores del Servicio LLM (Servidor). Si ocurre una falla en la comunicación con la API de OpenAI, el sistema debe capturar la excepción y devolver un mensaje de error estructurado con código HTTP 500 (Internal Server Error) al backend, indicando que el servicio de clasificación no está disponible en ese momento.

Casos de uso.

CU1: Clasificar temperamento de raza y traducir a español.

Actores: Backend de Adogta.

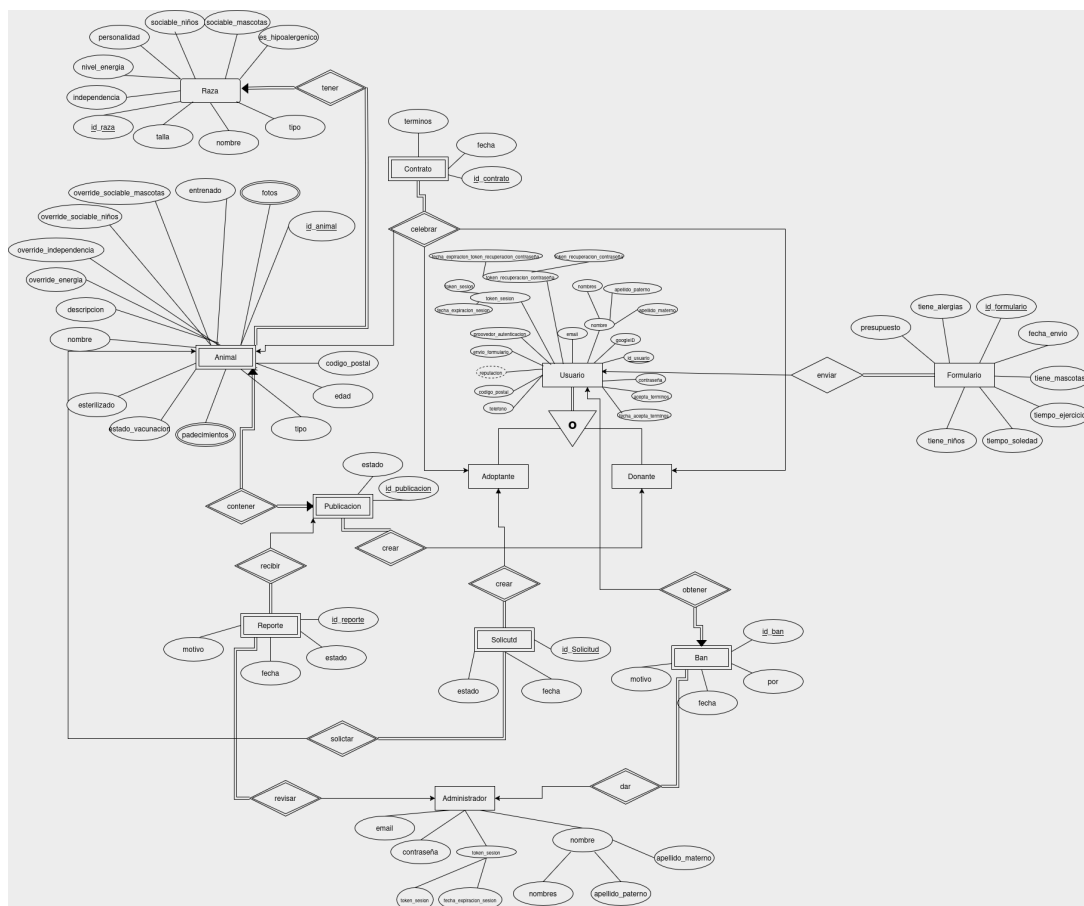
Descripción: El backend envía los datos obtenidos de The Dog API (nombre y temperamento en inglés de una raza) al microservicio para su respectiva traducción al español y cálculo del nivel de sociabilidad con niños, sociabilidad con otras mascotas, nivel de energía e independencia en un rango de 1 a 5.

Precondiciones:

- El backend de Spring Boot ha recolectado de forma exitosa los datos del animal.
- El microservicio se encuentra en ejecución con una API Key de OpenAI válida y operativa.

Flujo principal:

1. El backend emite una petición POST al endpoint /clasificar-temperamento.



La actualización que se hace para la versión 3 de la base de datos es agregar atributos a la entidad administrador para poder seguir su flujo de login y modificar constraints a la tabla publicación para poder seguir un flujo adecuado de los procesos de adopción.

Requerimientos no funcionales.

Se incluyen los siguientes requerimientos no funcionales que son aplicables a las funcionalidades implementadas en la Iteración 4.

RNF1: Seguridad de contraseñas

Las contraseñas de los usuarios deberán almacenarse mediante un algoritmo de hashing, de modo que no sean recuperables en texto plano bajo ninguna circunstancia.

RNF2: Protección de funcionalidades con token de expiración de sesión

El acceso a las funcionalidades del sistema deberá estar protegido mediante sesiones autenticadas con tiempo de expiración definido.

RNF3: Privacidad de usuarios y animales registrados

El sistema no deberá exponer la ubicación exacta de los usuarios ni de los animales registrados; la referencia geográfica se limitará al código postal.

RNF4: Privacidad de la información de los usuarios

El sistema no deberá mostrar directamente el correo electrónico de los usuarios entre sí; la comunicación inicial deberá canalizarse a través de los mecanismos de la plataforma.

RNF5: Rendimiento de búsqueda y filtrado

Las búsquedas y el filtrado de animales deberán devolver resultados en un tiempo promedio no mayor a 3 segundos bajo condiciones normales de operación.

RNF6: Procesamiento de imágenes rápido

Las fotografías de los animales deberán procesarse y almacenarse de forma que su carga en la ficha del animal no supere los 5 segundos bajo condiciones normales de red.

RNF9: Registro rápido e intuitivo

Un usuario sin experiencia previa en la plataforma deberá ser capaz de completar el registro y formulario de características del perfil en un máximo de 7 minutos.

RNF10: Sistema con soporte para múltiples usuarios concurrentes

El sistema deberá soportar el acceso concurrente de múltiples usuarios sin degradación perceptible en los tiempos de respuesta.

Limitaciones Actuales

La Iteración 4 representa el funcionamiento completo del sistema con flujo end-to-end (frontend + backend + microservicio + base de datos)

Funcionalidades pendientes para iteraciones futuras

- Registro mediante cuenta de Google (OAuth 2.0).
- Proceso formal de adopción con firma

Restricciones técnicas de la iteración

- La autenticación por Google OAuth se encuentra en fase de diseño; únicamente está disponible el registro con credenciales locales.
- Las imágenes de los animales se almacenan en el sistema de archivos local del servidor. En un entorno de producción esto debería migrarse a un servicio de almacenamiento en la nube.
- El CORS está configurado únicamente para `http://localhost:5173`, por lo que el sistema no está preparado para un despliegue en producción sin modificar esta configuración.
- La integración con Google Maps API requiere una clave de API válida; en entorno de desarrollo se usa una clave de prueba.
- El proceso de adopción (inicio y cierre) no incluye aún un mecanismo de firma digital; el contrato se envía para firma física.

Para el microservicio

- Dependencia de API Key de paga: El sistema depende de una clave de API activa de OpenAI para funcionar. Si los créditos se agotan, o la suscripción no es válida, las llamadas fallarán.
- Latencia esperada por la consulta al LLM: Dado que se realiza una petición de red a un modelo generativo grande, la latencia es sustancialmente mayor que la de un endpoint común. El backend de Spring Boot debe manejar tiempos de espera (timeouts) prolongados o implementar un procesamiento asíncrono para no bloquear los hilos principales ni empeorar la experiencia del usuario final.